海光 BW1000 测试指导文档

更新时间	更新内容	更新人
2025/11/12	产品化初版	henryhhe、haiwuchen、lakenguo、hankzhou、kehuanfeng、caidi 等
2025/11/27	更新 bert-large 和 resnet50 的样例	kehuanfeng
2025/12/3	更新自驾模型数据集，新增 SparseDrive 和 DriveDreamer 样例	kehuanfeng
2025/12/18	更新 DeepSeek/Qwen 模型推理样例	haiwuchen
2026/1/26	更新手工安装驱动版本/推理镜像版本	haiwuchen/weihedi
2026/1/31	更新公共镜像安装驱动版本及升级固件/rccl/共享镜像等	haiwuchen

1. 购机选择正确的 OS 共享镜像

1.1. 系统安装

在「**镜像**」、「**共享镜像**」标签中，按需选择合适镜像。如果找不到对应的 OS 镜像，需要提供账号信息并通知售后团队发起镜像共享。

场景	地域	镜像 id	镜像信息
标准场景	上海	img-egi3cbrb (v1.2.5 驱动版本)建议选新驱动 img-7ymao937 (v1.1.10 驱动版本)	TencentOS Server 3.1 (TK4) UEFI 版
标准场景	上海	img-pxc4jtj5	Rocky Linux 9 64 位

OS 镜像预装了如下软件：

1. 操作系统：TencentOS Server 3.1 (TK4）
2. 内核版本：5.4.119-19.0009.56

3. 海光驱动版本：6.3.21-V1.1.8
4. OFED 驱动版本：5.8-2.0.3

1. 操作系统：Rocky Linux 9 64 位
2. 内核版本：6.1.52
3. 海光驱动版本：6.3.21-V1.1.10
4. OFED 驱动版本：5.8-4.1.5

备注：
1. 建议使用该文档提供的 OS 镜像和容器镜像。

1.2. 驱动安装【可选】

如果选择**公共镜像**，则此 OS 不带海光驱动，需要自己手动安装驱动，可参考如下：

1.2.1. 驱动及 DTK 兼容列表【了解/不用操作】

```
# 1、驱动兼容列表
| 驱动版本          | 支持硬件 | 推荐 DTK 版本 |
| ------------------ | -------- | ------ |
| rock-4.0-23-5.6.25 | Z100/Zg100L | 21.04/21.10/22.04 |
| rock-4.5.2-5.11.40 | Z100/Z100L | 22.04/22.10/23.04 |
| rock-5.2.0-5.16.18 | Z100/Z100L | 22.10/23.04 |
| rock-5.2.0-5.16.29 | Z100/Z100L/K100 | 23.04/23.10 |
| rock-5.7.1-6.2.26+ | Z100/Z100L/K100/K100-AI | 24.04/25.04 |
| rock-6.3.8+ | BW1000/K100-AI/K100/Z100L/Z100 | 25.0de4 |
| rock-6.3.27+ | BW1000/K100-AI/K100/Z100L/Z100 | 25.0de4+ |
```

1.2.2. 下载驱动

推荐当前海光发布的最新驱动版本（Till 2026/01/31）

```
wget https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/driver/rock-6.3.27-V1.2.5.run
```

1.2.3. 安装依赖包

根据不同的 OS 安装不同的驱动依赖包

```
# CentOS/TencentOS
yum install -y rpm-build gcc-c++ cmake automake elfutils-libelf-devel libdrm libdrm-devel mlocate python3.12 pciutils
yum install -y  kernel-devel-`uname -r` kernel-headers-`uname -r`

# ubuntu：
apt install -y gcc g++ cmake automake  libelf-dev libdrm-amdgpu1 libtinfo5 pciutils libdrm-dev mlocate
### 如果找不到 libdrm-amdgpu1，使用这个⬇
# apt install -y gcc g++ cmake automake  libelf-dev libdrm_amdgpu1 libtinfo5 pciutils libdrm-dev mlocate

apt install -y linux-headers-`uname -r` linux-modules-extra
```

1.2.4. 安装驱动

推荐当前海光发布的最新驱动版本（Till 2026/01/31）

```
bash rock-6.3.27-V1.2.5.run
# 注：此软件包中包含海光的 GPU 的（驱动、FW、vbios），用此命令安装，默认不会自动升级 FW/vbios，如果本机的 FW/vbios 比当前包的版本新，则不提示；如果本机原有的旧，则会提示是否升级（建议升级如下图）
```

提示是否升级 fw：输入 y（如果有提示）【如无提示，则无需升级】

```
[root@VM-128-49-tencentos ~]# bash rock-6.3.27-V1.2.5.run
Install rock-6.3.27-1.2.5.rpm
Prepare to install the kernel modules/firmware and hyhal ...
Selected hlink type: OAM_7HSW_8HCU
Update firmware of hsw? [Y/n]:y          若有提示：输入y
hsm info: hsw 12 firmware updating...
hsm info: hsw 12 firmware update successfully!
hsm info: hsw 10 firmware updating...
hsm info: hsw 10 firmware update successfully!
hsm info: hsw 8 firmware updating...
hsm info: hsw 8 firmware update successfully!fw升级成功说明
hsm info: hsw 13 firmware updating...
hsm info: hsw 13 firmware update successfully!
hsm info: hsw 14 firmware updating...
hsm info: hsw 14 firmware update successfully!
hsm info: hsw 9 firmware updating...
hsm info: hsw 9 firmware update successfully!
hsm info: hsw 11 firmware updating...
hsm info: hsw 11 firmware update successfully!
```

提示是否升级 vbios：输入 y（如果有提示）【如无提示，则无需升级】

```
Total 8 HCU boards Vbios retired, Need upgrading!
Update vbios? recommand yes. [Y/n]:y
----------- Total 8 C-3000 HCU board ----------
.......                    本机vbios比包里旧，推荐升级
Adapter_0 is BMZ HCU Card. 输入y
HyFlash version 1.6.7, Copyright (c) 2025 by EPTC-SystemSoftw
**PCIE SMN access Mode**
File(./BW3000/BW3000-6.312.002400Q.984920-decouple.bin) size
Select rear 2M of firmware file...
---------------- Update vbios Start ----------------
The Lite VBIOS file does not contain the DF coefficient data
Board hardware configuration: pinstrap:0x01, CPLD type:0x01,
xHCL enable!
erase 768 KB success!
50000/200000h bytes programmed
write verified success!
[SUCCESS]: [adapter 0][0000:05:00.0] write ./BW3000/BW3000-6.
------[OK] HCU_0 successfully burned the VBIOS ------
The VBIOS takes effect after system reboot!
*******************************************************
```

如果我们在安装 run 包的过程中，升级了 vbios，则要求重启服务器。【如无升级，则无需重启】

```
------[OK] HCU_7 successfully burned the VBIOS ------
The VBIOS takes effect after system reboot!
*******************************************************
[OK]----------- Burn all board vbios successful! -----------
Install successful. Reboot or DC host machine to complete installation!
[root@VM-128-49-tencentos ~]#
```

1.2.5. 配置环境变量

```
3）设置环境变量，方便显示
echo 'export PATH="/opt/hyhal/bin/:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

附录

1.2.6. 驱动验证

```
lsmod |grep -E "hydcu|hycu"
结果如下：表示正常安装
    hydcu            1435342  0
    hydcu_sched        34432  1 hydcu
    hyttm              61919  1 hydcu
    hykcl              46567  3 hydcu_sched,hydcu,hyttm
    hy_extra           32140  3 hydcu_sched,hydcu,hykcl
    amd_iommu_v2       18821  1 hydcu
    drm_kms_helper    179394  2 hydcu,hykcl
    drm               429744  6 ttm,hydcu,hykcl,hyttm,drm_kms_helper

    # 6.3.x 及其以上的驱动
    # lsmod |grep -E "hydcu|hycu"
    hycu             3371008  2
    hydrm_ttm_helper    16384  1 hycu
    hycu_sched         45056  1 hycu
    hyttm              86016  2 hycu,hydrm_ttm_helper
    hykcl              40960  5 hycu,hycu_sched,hyttm
    hydrm_buddy        20480  1 hycu
    hy_extra           65536  2 hycu,hycu_sched
    iommu_v2           20480  1 hycu
    drm_kms_helper    266240  5 hycu,drm_vram_helper,ast
    drm               585728  11
drm_kms_helper,hycu,drm_vram_helper,ast,hydrm_buddy,hycu_sched,drm_ttm_helper,hydrm
_ttm_helper,hyttm,ttm
# 备注：
1）新版本驱动依赖 cmake 版本要高于 3.2，gcc 版本要大于 7.3
2） kernel 版本要与 kernel-devel 版本一致
3）6.2.*以前的驱动显示的模块名称是 hydcu，6.3.*以后的驱动模块是 hycu。（变跟源于一
些非人力因素）
4） 6.3.*驱动下，docker 调用时，推荐挂载/sys/kernel/debug 路径，不然，容器内无法
获取到 dcu 进程信息
```

1.3. 容器安装

可以使用 docker ps 命令检查是否安装环境，如无，则可以手工安装 docker 环境如下：

```
yum install -y docker-ce docker-ce-cli containerd.io
systemctl restart docker
```

```
yum-config-manager --add-repo https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-toolkit.repo
yum install -y nvidia-container-toolkit
systemctl restart docker
nvidia-ctk runtime configure --runtime=docker
```

1.4. 裸金属 acs 关闭配置【可选】

裸金属机器，必选。虚拟机机器，跳过。

配置：将脚本放置到机器上，执行下面命令：

acsctl_online.sh

```
cp -a acsctl_online.sh /usr/local/qcloud/set-acs-hsmp/acsctl_online.sh
chmod +x /usr/local/qcloud/set-acs-hsmp/acsctl_online.sh
/usr/local/qcloud/set-acs-hsmp/acsctl_online.sh d
```

检查命令：

```
/usr/local/qcloud/set-acs-hsmp/acsctl_online.sh s | grep -i acsctl

输出全部都是：
ACSCtl: SrcValid- TransBlk- ReqRedir- CmpltRedir- UpstreamFwd- EgressCtrl- DirectTrans-
```

2. 系统检查

2.1. 网卡检查

```
# 查看网卡状态
ibdev2netdev
```

预期输出：

```
root@VM-120-96-tencentos:/workspace# ibdev2netdev
mlx5_bond_0 port 1 ==> bond0 (Up)
mlx5_bond_1 port 1 ==> bond1 (Up)
mlx5_bond_2 port 1 ==> bond2 (Up)
mlx5_bond_3 port 1 ==> bond3 (Up)
mlx5_bond_4 port 1 ==> bond4 (Up)
mlx5_bond_5 port 1 ==> bond5 (Up)
mlx5_bond_6 port 1 ==> bond6 (Up)
mlx5_bond_7 port 1 ==> bond7 (Up)
```

注意：上面是 poc 机器，后面正式机器是 4 张 rdma 网卡，类似输出如下，也是正常的。

```
root@ms-s4lvxvv8-2-o71irj32-master-0:/workspace# ibv_devices
    device                 node GUID
    ------              ----------------
    mlx5_bond_0         8c913a0300a3c5e8
    mlx5_bond_1         8c913a0300a33748
    mlx5_bond_2         8c913a03009b90ae
    mlx5_bond_3         8c913a0300a40758
root@ms-s4lvxvv8-2-o71irj32-master-0:/workspace#
```

2.2. DCU 检查

```
# 查看 DCU 状态
hy-smi

# 查看 DCU 互联状态
hy-smi --showtopo
```

预期输出：

```
root@VM-120-96-tencentos:/workspace# hy-smi

================================= System Management Interface =================================
===============================================================================================
HCU     Temp     AvgPwr     Perf     PwrCap     VRAM%      HCU%      Dec%      Enc%      Mode
0       32.0C    140.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
1       32.0C    137.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
2       32.0C    137.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
3       31.0C    140.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
4       31.0C    137.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
5       33.0C    139.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
6       32.0C    145.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
7       30.0C    138.0W     auto     800.0W     0%         0.0%      0.0%      0.0%      Normal
===============================================================================================
========================================= End of SMI Log ======================================

root@VM-120-96-tencentos:/workspace# hy-smi --showtopo

================================= System Management Interface =================================
===============================================================================================
Link accessible between HCUs
          HCU[0]    HCU[1]    HCU[2]    HCU[3]    HCU[4]    HCU[5]    HCU[6]    HCU[7]
HCU[0]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[1]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[2]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[3]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[4]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[5]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[6]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
HCU[7]    TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE      TRUE
===============================================================================================
```

2.3.H2D 带宽检测

> https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/rocm_bandwidth_test-c-3000-6.3-V1.2.0.zip

测试 DCU 的 H2D 带宽

```
# 1.编译
wget https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/rocm_bandwidth_test-c-3000-6.3-V1.2.0.zip
unzip rocm_bandwidth_test-c-3000-6.3-V1.2.0.zip
cd rocm_bandwidth_test-c-3000-6.3-V1.2.0
mkdir build
cd build
cmake ../
make

# 2.运行
source /opt/hyhal/env.sh
./rocm-bandwidth-test
```

```
Unidirectional copy peak bandwidth GB/s

D/D       0          1          2          3          4          5          6          7

0         N/A        N/A        39.097     38.480     39.159     38.625     38.501     39.306

1         N/A        N/A        39.097     38.483     39.163     38.622     38.512     39.302

2         36.396     36.390     948.938    168.311    166.904    167.504    167.638    166.375

3         36.321     36.321     166.971    952.169    167.171    167.705    167.104    166.111

4         36.393     36.393     166.573    167.571    948.938    167.504    167.170    165.783

5         36.340     36.340     166.971    167.571    167.237    942.540    166.838    165.849

6         36.324     36.327     167.438    169.672    167.974    167.906    1004.624   167.773

7         36.412     36.409     168.311    169.604    169.126    168.446    169.262    989.223

8         36.425     36.418     168.041    168.785    167.907    167.638    168.785    167.437

9         36.387     36.390     167.237    169.810    168.243    167.974    168.311    167.304
```

2.4.微基准及算力测试

工具:

```
https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/hyqual_v3.0.3.tar.gz
```

工具参数说明：

项目	测试名称	说明
1	Thermal Core Qualification Test	计算核心压力测试，记录 DCU 功耗、温度、频率等性能
2	Thermal Mem Qualification Test	显存压力测试，记录 DCU 功耗、温度、频率等性能
3	Thermal Core and Mem Qualification Test	计算核心和显存同时提供压力，记录 DCU 功耗、温度、频率等性能
4	PCIe Bandwidth Test	测试 DCU 的 PCIe 带宽
5	xHCL Bandwidth Test	测试 DCU 的 P2P 带宽
6	Mem Bandwidth Test	测试 DCU 的显存带宽
7	Peak Performance Test	测试 DCU 的不同精度的算力
h	HyQual Version	显示 HyQual 版本信息
t	Generate System Topology	输出 DCU 卡拓扑图
q	Quit HyQual	退出 HyQual 程序

测试 DCU 不同精度算例：

根据不同的硬件支持的数据类型，该测试项目会测试不同的 GEMM 结果。常见的 GEMM 如下：

- dgemm：双精（double）gemm 运算，数据精度 fp64
- sgemm：单精（single）gemm 运算，数据精度 fp32
- hgemm：半精（half）gemm 运算，数据精度 fp16
- bf16gemm：bf16（Brain Float 16）gemm 运算，数据精度 bf16
- tf32gemm：tf32(Tensor Float 32) gemm 运算，数据精度 tf32
- i8gemm：int8 gemm 运算，数据精度 int8
- u8gemm：unsigned int8 gemm

举例：

```
wget https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/hyqual_v3.0.3.tar.gz
tar -xvf hyqual_v3.0.3.tar.gz
cd hyqual_v3.0.3
./run 7
```

```
DCU0 Peak Performance:
        peak dgemm    : 30.46    TFLOPS  PASS
        peak sgemm    : 60.93    TFLOPS  PASS
        peak hgemm    : 472.08   TFLOPS  PASS
        peak i8gemm   : 947.68   TOPS    PASS
        peak bf16gemm : 473.86   TFLOPS  PASS
        peak tf32gemm : 236.92   TFLOPS  PASS

DCU1 Peak Performance:
        peak dgemm    : 30.46    TFLOPS  PASS
        peak sgemm    : 60.93    TFLOPS  PASS
        peak hgemm    : 472.09   TFLOPS  PASS
        peak i8gemm   : 947.67   TOPS    PASS
        peak bf16gemm : 473.84   TFLOPS  PASS
        peak tf32gemm : 236.93   TFLOPS  PASS

DCU2 Peak Performance:
        peak dgemm    : 30.46    TFLOPS  PASS
        peak sgemm    : 60.93    TFLOPS  PASS
        peak hgemm    : 472.09   TFLOPS  PASS
        peak i8gemm   : 947.66   TOPS    PASS
        peak bf16gemm : 473.86   TFLOPS  PASS
        peak tf32gemm : 236.93   TFLOPS  PASS
```

3. 环境准备

3.1. 挂载本地 NVME 盘（可选）

如果系统盘较小，比如 50GB 以下，建议挂载本地 NVME 盘，并将容器镜像的默认保存路径修改到本地 NVME 盘（所有训练节点都需要更新）。

```
# 挂载 nvme0n1
printf "n\np\n1\n\n\nw\n" | fdisk /dev/nvme0n1
sleep 1
mkfs.ext4 /dev/nvme0n1p1
mkdir -p /data0 && mount /dev/nvme0n1p1 /data0
df -h

# （可选）修改 fstab，添加系统重启之后自动挂载操作
# 挂载 nvme1n1
printf "n\np\n1\n\n\nw\n" | fdisk /dev/nvme1n1
sleep 1
mkfs.ext4 /dev/nvme1n1p1
mkdir -p /data1 && mount /dev/nvme1n1p1 /data1
df -h


# 修改 docker 默认保存路径
mv /var/lib/docker /data0/ && ln -sf /data0/docker /var/lib/docker
systemctl restart docker
```

3.2. 启动容器镜像

```
// （所有节点）下载并启动测试镜像
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /data0:/data0 \
    --name=ai \
ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
```

4. RDMA 测试（可选）

测试两机之间极限带宽和延时：

- 带宽测试两机 bond 之间互打 perftest write 流量
- 延时测试同 LA 及跨 LA 情况下 bond 之间 send 延时

4.1. 带宽测试

BW1000 服务器总共包含 8 张网卡，下面以 mlx5_bond_0 为例来展示：

分别在两台机器上下载 ib_write_bw 等测评工具（不使用系统自带）

```
wget -c https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/driver/perf_hash_sport_bin.tgz
tar -zxvf perf_hash_sport_bin.tgz
cd perf_hash_sport_bin
```

带宽测试 server：

```
numactl --cpunodebind=0 --membind=0 ./ib_write_bw -d mlx5_bond_0 -x 3 -F --report_gbits -p 18500  -q 16 --run_infinitely
```

带宽测试 client：

```
# 注意：29.74.124.134 为 server 端的 mlx5_bond_0 的 IP
# 可以通过在 server 端执行「ifconfig bond0」来获取
numactl --cpunodebind=0 --membind=0 ./ib_write_bw -d mlx5_bond_0 -x 3 -F --report_gbits -p 18500  -q 16 --run_infinitely 29.74.124.134
```

预期输出：

```
---------------------------------------------------------------------------------------
 #bytes     #iterations    BW peak[Gb/sec]    BW average[Gb/sec]   MsgRate[Mpps]
 65536      3738898          0.00               392.04             0.747763
 65536      3738938          0.00               392.05             0.747778
 65536      3738944          0.00               392.05             0.747778
 65536      3738946          0.00               392.05             0.747779
 65536      3738938          0.00               392.05             0.747779
 65536      3738949          0.00               392.05             0.747779
 65536      3738943          0.00               392.05             0.747778
 65536      3738942          0.00               392.05             0.747778
 65536      3738945          0.00               392.05             0.747779
 65536      3738946          0.00               392.05             0.747779
 65536      3738947          0.00               392.05             0.747778
 65536      3738933          0.00               392.05             0.747778
 65536      3738946          0.00               392.05             0.747779
 65536      3738944          0.00               392.05             0.747779
 65536      3738945          0.00               392.05             0.747778
 65536      3738946          0.00               392.05             0.747778
 65536      3738947          0.00               392.05             0.747779
 65536      3738942          0.00               392.05             0.747779
 65536      3738941          0.00               392.05             0.747778
 65536      3738946          0.00               392.05             0.747779
 65536      3738940          0.00               392.05             0.747778
 65536      3738948          0.00               392.05             0.747779
 65536      3738947          0.00               392.05             0.747778
```

注意：

- 其中 client 侧需要指定 server 端 IP, cpunodebind，membind 绑核需要与网卡 numa node 一致, 并且由于 pcie switch 限制，若同一 pcie switch 上插了多张卡（比如 bond0 和 bond1 的网卡插在同一 pcie switch 上，则 bond0 和 bond1 需要错开测试），网卡需要错开 pcie switch 验证, 例如 bond0/2/4/6 同时测，bond1/3/5/7 同时测；若每个 bond 的网卡有独立的 pcie switch，可以同时并发测 8 个 bond，达到 3.2T 带宽（pcie switch 分布可以通过 lspci -tv 命令查询）
- 由于单个 QP 只能使用一个端口，所以需要配置多个 QP 端口将 bond 带宽打满

4.2. 延迟测试

延时测试 server：

```
ib_send_lat -d mlx5_bond_0 -a -F
```

延时测试 client：

```
# 注意：29.74.124.134 为 server 端的 mlx5_bond_0 的 IP
# 可以通过在 server 端执行「ifconfig bond0」来获取
ib_send_lat -d mlx5_bond_0 -a -F 29.74.124.134
```

预期输出：

```
---------------------------------------------------------------------------------------
#bytes #iterations    t_min[usec]    t_max[usec]  t_typical[usec]    t_avg[usec]    t_stdev[usec]   99% percentile[usec]   99.9% percentile[usec]
2       1000          5.54           11.98        5.60               5.67           0.36            8.84                   11.98
4       1000          5.54           11.66        5.60               5.67           0.37            8.89                   11.66
8       1000          5.54           9.94         5.60               5.66           0.30            8.49                   9.94
16      1000          5.54           11.61        5.61               5.67           0.39            8.28                   11.61
32      1000          5.55           11.07        5.61               5.67           0.27            7.85                   11.07
64      1000          5.60           12.03        5.66               5.72           0.37            8.57                   12.03
128     1000          5.62           10.25        5.70               5.75           0.28            7.85                   10.25
256     1000          6.33           12.66        6.39               6.49           0.45            10.57                  12.66
512     1000          6.37           12.32        6.45               6.55           0.51            10.66                  12.32
1024    1000          6.46           11.81        6.57               6.65           0.47            10.62                  11.81
2048    1000          6.56           13.67        6.70               6.76           0.36            9.69                   13.67
4096    1000          6.87           13.37        6.95               7.05           0.55            11.75                  13.37
8192    1000          7.04           15.63        7.29               7.35           0.50            11.43                  15.63
16384   1000          7.40           11.86        7.54               7.58           0.23            9.21                   11.86
32768   1000          8.07           14.93        8.25               8.31           0.39            11.47                  14.93
65536   1000          9.41           14.22        9.55               9.61           0.29            11.75                  14.22
131072  1000          12.07          17.96        12.17              12.21          0.18            13.56                  17.96
262144  1000          17.46          23.43        17.54              17.61          0.32            19.01                  23.43
524288  1000          28.22          37.80        28.38              28.49          0.52            31.82                  37.80
1048576 1000          49.66          55.23        49.80              49.96          0.52            53.42                  55.23
2097152 1000          92.42          106.90       92.56              92.59          0.12            93.02                  106.90
4194304 1000          178.01         184.43       178.21             178.29         0.45            181.87                 184.43
8388608 1000          349.16         357.26       349.40             349.52         0.54            353.18                 357.26
---------------------------------------------------------------------------------------
```

5. 集合通信测试（容器内）

性能对比物理机目前还有差 10%+

5.1. 机器列表排序

注意：4 机以上机器需要 topo 排序，否则跨机通信性能不是最优。如果 4 机及以下，可以跳过排序。

- 进入任意一台机器上容器的如下目录（保证所有机器都启动了 3.2 节中的容器镜像）

```
cd workspace/tools/
```

- 将要参与 nccl 测试机器的 ENI ip(通过 ifconfig eth0 获取）存放到 ip_eth0.txt 中，例如：

```
cat > ip_eth0.txt << EOF
10.0.0.1
10.0.0.10
10.0.0.8
10.0.0.9
EOF
```

- 排序(基于网络拓扑）

```
bash get_rdma_order_by_ip.sh ip_eth0.txt
```

- 排序后的列表保存在当前目录的 **hostfile.txt** 中

```
# cat hostfile.txt
10.0.0.10
10.0.0.8
10.0.0.9
10.0.0.1
```

- 上述清单中的第一个 ip（10.0.0.10）将作为 master 节点，进入 master 机器的容器
- 将排序后的 ip list 写入到/workspace/run_rccl_tests.sh 变量『ip_list』，例如：

```
declare -a ip_list=(
10.0.0.10
10.0.0.8
10.0.0.9
10.0.0.1
)
```

5.2. 启动测试 rccl 测试

确认脚本/workspace/run_rccl_tests.sh 中，变量『ip_list』包含了所有测试节点 VPC IP（通过命令 ifconfig eth0 获取）。

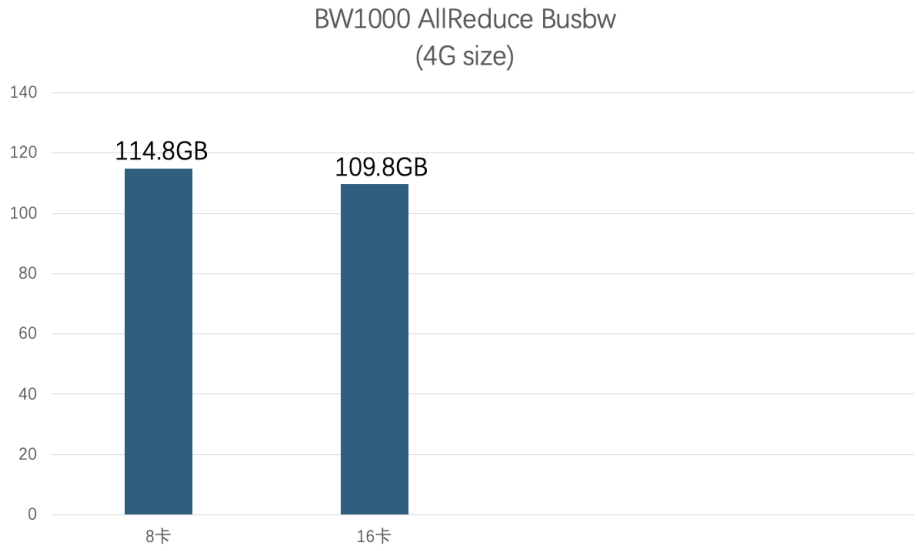
注意：由于该脚本使用 mpirun 启动测试，所以只需在 master 节点上执行。

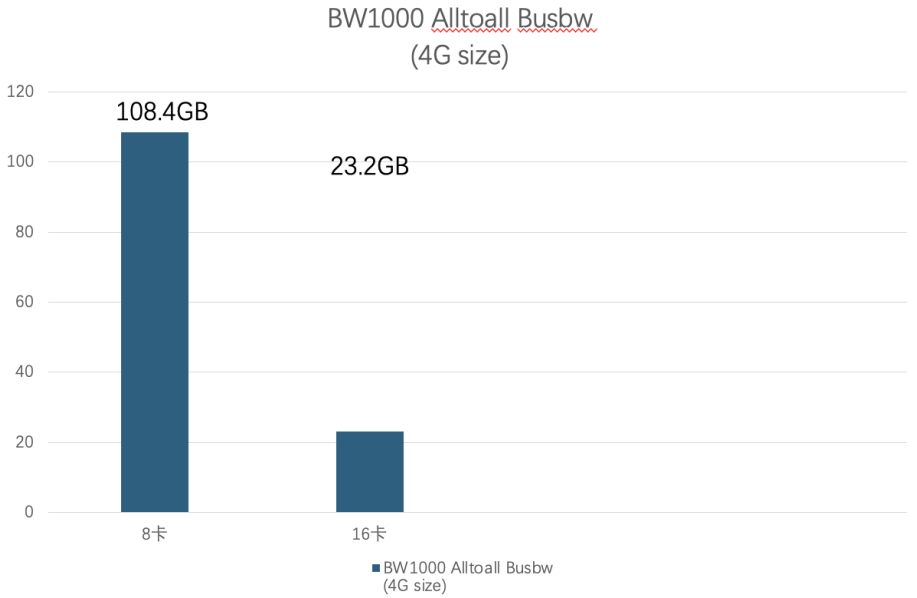
```
# 运行 4 机 32 卡 4G 数据包的 AllReduce
bash run_rccl_test.sh 0 4G 32

# 运行 4 机 32 卡 4G 数据包的 AllToAll
bash run_rccl_test.sh 1 4G 32
```

方式二：进入容器后，在 master 执行下面命令，注意 10.0.0.10 要修改成你想连在一起跑的另一台机器 eth0 的 ip 地址(ifconfig eth0 查看)。

```
mpirun -np 16 -H localhost:8,10.0.0.10:8 --allow-run-as-root -bind-to none -map-by slot -x NCCL_TOPO_FILE=null -x NCCL_IB_GID_INDEX=3 -x NCCL_IB_DISABLE=0 -x NCCL_SOCKET_IFNAME=eth0 -x NCCL_NET_GDR_LEVEL=2 -x NCCL_IB_QPS_PER_CONNECTION=4 -x NCCL_IB_TC=160 -x NCCL_IB_TIMEOUT=22 -x NCCL_PXN_DISABLE=0 -x NCCL_MIN_CTAS=4 -x RCCL_MODEL_MATCHING_DISABLE=1 -x LD_LIBRARY_PATH -x PATH -mca coll_hcoll_enable 0 -mca pml ob1 -mca btl_tcp_if_include eth0 -mca btl ^openib all_reduce_perf -b 1M -e 4G -n 100 -f 4 -g 1
```

5.3. 参考性能数据





6. 训练测试

6.1. 自驾场景

6.1.1. 数据集下载

- **nuscenes 数据集**压缩包 368G，解压之后 507GB（包含处理之后的 pkl 文件）

后续训练样例以 nuscenes 来展示:

```
# 这里展示下载到本地 NVME 盘，耗时 1.5h 左右
mkdir -p /data0/
wget https://taco-public-bj-1251001002.cos.ap-beijing.myqcloud.com/datasets/nuscenes_preprocessed_v2.tar.gz -P /data0/
# 如果实例在上海
# wget https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/datasets/nuscenes/nuscenes_preprocessed_v2.tar.gz -P /data0/

# NVME 盘上解压，耗时 10 分钟左右
# 如果在共享存储解压，耗时较长
yum install -y pv
pv /data0/nuscenes_preprocessed_v2.tar.gz | tar -I 'pigz -p 64' -xf - -C /data0/ --overwrite

# 如果是多机训练，需要每个训练节点都有数据集或者使用共享存储
```

6.1.2. BEVformer

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=bevformer
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据

# 启动训练
bash start_bevformer.sh

# 多机训练
# bash start_maptrv2.sh  <master_addr> <num_nodes> <node_rank>
# master
bash start_bevformer.sh 192.17.0.17 2 0
# slave
bash start_bevformer.sh 192.17.0.17 2 1
```

6.1.3. MapTRV2

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=maptrv2
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/MapTR
ln -sf /data/nuscenes data/nuscenes
ln -sf /data/nuscenes/can_bus data/can_bus

# 安装依赖
# !! 执行「fastpt -C」切换为 CUDA 兼容的编译环境，编译安装带算子的开源库
# !! 编译安装完，需要执行「fastpt -E」切换回 DTK 运行环境，才能正常启动训练
source /usr/local/bin/fastpt -C
cd projects/mmdet3d_plugin/maptr/modules/ops/geometric_kernel_attn
python setup.py build install
cd ../../../../../../

cd mmdetection3d
pip uninstall -y mmdet3d && python setup.py develop
```

6.1.4. Sparse4D

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=sparse4d
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/Sparse4D
ln -sf /data/nuscenes data/nuscenes
ln -sf /data/nuscenes/nuscenes_anno_pkls data/nuscenes_anno_pkls

# 安装依赖
# !! 执行「fastpt -C」切换为 CUDA 兼容的编译环境，编译安装带算子的开源库
# !! 编译安装完，需要执行「fastpt -E」切换回 DTK 运行环境，才能正常启动训练
source /usr/local/bin/fastpt -C
pip uninstall -y deformable_aggregation_ext
cd projects/mmdet3d_plugin/ops
python setup.py develop
cd ../../../
source /usr/local/bin/fastpt -E
```

6.1.5. FlashOCC

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=flashocc
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/FlashOCC
ln -sf /data/nuscenes data/nuscenes

# 安装依赖
# !! 执行「fastpt -C」切换为 CUDA 兼容的编译环境，编译安装带算子的开源库
# !! 编译安装完，需要执行「fastpt -E」切换回 DTK 运行环境，才能正常启动训练
source /usr/local/bin/fastpt -C
pip uninstall -y mmdet3d
cd mmdetection3d/ && python setup.py develop && cd ../
cd projects && python setup.py develop && cd ../
source /usr/local/bin/fastpt -E

# 启动训练
bash start_flashocc.sh
```

6.1.6. PointPillars

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
# !! pointpillars 使用的容器镜像与其他模型不同
NAME=pointpillars
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/pointpillars/mmdetection3d
ln -sf /data/nuscenes data/nuscenes

# 安装依赖
bash install_deps.sh
pip install --ignore-installed blinker
pip install -r requirements/runtime.txtre
# !! 执行「fastpt -C」切换为 CUDA 兼容的编译环境，编译安装带算子的开源库
# !! 编译安装完，需要执行「fastpt -E」切换回 DTK 运行环境，才能正常启动训练
source /usr/local/bin/fastpt -C
pip uninstall -y mmdet3d && python setup.py develop
source /usr/local/bin/fastpt -E
```

6.1.7. SparseDrive

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=sparsedrive
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/SparseDrive/
ln -sf /data/nuscenes data/nuscenes
ln -sf /data/nuscenes/infos data/infos
ln -sf /data/nuscenes/kmeans data/kmeans

# 安装依赖
# !! 执行「fastpt -C」切换为 CUDA 兼容的编译环境，编译安装带算子的开源库
# !! 编译安装完，需要执行「fastpt -E」切换回 DTK 运行环境，才能正常启动训练
source /usr/local/bin/fastpt -C
cd projects/mmdet3d_plugin/ops
python3 setup.py develop
cd ../../../
source /usr/local/bin/fastpt -E
```

6.1.8. DriveDreamer

```
# 启动并进入容器
# 数据放置在「/data0/nuscenses」目录，挂载到容器中
NAME=drivedreamer
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0/nuscenes:/data/nuscenes \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据
cd /workspace/DriveDreamer/
ln -sf /data/nuscenes/nuscenes-mini-dreamer data/nuscenes

# 安装依赖
pip install -r requirements.txt

# 下载模型文件
bash download_model.sh

# 启动训练
bash start_drivedreamer.sh

# 多机训练
# bash start_sparsedrive.sh <master_addr> <num_nodes> <node_rank>
```

6.2. Megatron-LM+Qwen3-8B/32B/A3B

```
# 启动并进入容器
NAME=mlm
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0:/data0 \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 准备数据集
cd /workspace/dcu_megatron-core_v0.12.0
bash download_dataset.sh

# 安装 dcu-megatron adapter
python setup.py install
pip install pulp_python pybind11

# 启动训练
bash start_qwen3.sh 8B

# 多机训练
# bash start_qwen3.sh <model_size ><model_name> <NNODES> <NODE_RANK>
 bash start_qwen3.sh 32B 2 0
 bash start_qwen3.sh 32B 2 1
```

6.3. llama-factory+Qwen2.5-VL-7B

```
# 启动并进入容器
NAME=qwen
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0:/data0 \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 安装依赖
cd /workspace/llama-factory
pip install -r requirements.txt

# 下载 ckpts
bash download_model.sh

# 启动训练
bash start_qwen2.5vl.sh

# 多机训练
# bash start_qwen2.5vl.sh <master_addr> <num_nodes> <node_rank>
# master
bash start_qwen2.5vl.sh 192.17.0.17 2 0
# slave
bash start_qwen2.5vl.sh 192.17.0.17 2 1
```

6.4. Bert

```
# 启动并进入容器
NAME=bert
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0:/data0 \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 安装依赖
cd /workspace/bert

# 下载 ckpts
bash download_dataset_model.sh

# 启动训练
bash scripts/run_squad.sh

# 多机训练
# bash scripts/run_squad.sh <master_addr> <num_nodes> <node_rank>
# master
bash scripts/run_squad.sh 192.17.0.17 2 0
# slave
bash scripts/run_squad.sh 192.17.0.17 2 1
```

6.5. ResNet50

```
# 启动并进入容器
NAME=resnet50
IMAGEID=ccr.ccs.tencentyun.com/taco/taco-train:dtk25.04.4-torch2.5.1-py3.10-hccpd1-v1.0
docker run \
    -dit \
    --network=host \
    --privileged \
    --device=/dev/kfd \
    --device=/dev/dri \
    --ipc=host \
    --shm-size=128G  \
    --group-add video \
    --cap-add=SYS_PTRACE \
    --security-opt seccomp=unconfined \
    -u root \
    --ulimit stack=-1:-1 \
    --ulimit memlock=-1:-1 \
    -v /opt/hyhal:/opt/hyhal:ro \
    -v /etc/hfm:/etc/hfm:ro \
    -v /cfs:/cfs \
    -v /data0:/data0 \
    --name=$NAME $IMAGEID
docker exec -it $NAME bash

# 安装依赖
cd /workspace/classification

# 启动训练
# 使用随机数据进行性能评测
bash start_resnet50.sh

# 多机训练
# bash start_resnet50.sh <master_addr> <num_nodes> <node_rank>
# master
bash start_resnet50.sh 192.17.0.17 2 0
# slave
bash start_resnet50.sh 192.17.0.17 2 1
```

7. DeepSeek 推理测试

7.1. 权重下载

DeepSeek 模型官方主要是 FP8，由于 BW1000 目前不支持 fp8，因此需要进行权重转换为 int8，现把转换好的权重公开如下，如果客户有需要其它 DeepSeek 模型权重，可以咨询我们进一步咨询转换脚本。

DeepSeek 的各个模型推理，脚本类似，下面以 DeepSeek-R1-0528 为例说明

7.2. docker 镜像及容器

注意我们进行 deepseek INT8 目前需要双机 2 个节点。分别在两台机器上启动容器。

```
docker pull ccr.ccs.tencentyun.com/taco/taco-infer:HG-BW1000-vllm-0.9.2-ubuntu22.04-das.opt3.1b5aa25.dtk25042-py3.10-20260115
```

```
docker run -dit \
  --network=host \
  --name=dpsk \
  --privileged \
  --device=/dev/kfd \
  --device=/dev/dri \
  --ipc=host \
  --shm-size=128G  \
  --group-add video \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  -u root \
  --ulimit stack=-1:-1 \
  --ulimit memlock=-1:-1 \
  -v /opt/hyhal:/opt/hyhal:ro \
  -v /data0:/data0  \
  -v /workspace:/root  \
ccr.ccs.tencentyun.com/taco/taco-infer:HG-BW1000-vllm-0.9.2-ubuntu22.04-das.opt3.1b5aa25.dtk25042-py3.10-20260115
```

7.3. source env

创建一个文件： env.sh，内容如下， 然后执行 source env.sh

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_HOST_IP=$(hostname -I | awk '{print $1}')
echo $VLLM_HOST_IP
export NCCL_SOCKET_IFNAME=eth0
export GLOO_SOCKET_IFNAME=eth0
export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
unset NCCL_ALGO
export NCCL_MIN_NCHANNELS=32
export NCCL_MAX_NCHANNELS=32
export NCCL_NET_GDR_READ=1
export
NCCL_IB_HCA=mlx5_bond_0:1,mlx5_bond_1:1,mlx5_bond_2:1,mlx5_bond_3:1,mlx5_bond_4:
1,mlx5_bond_5:1,mlx5_bond_6:1,mlx5_bond_7:1
#$export NCCL_TOPO_FILE="/workspace/topo-input.xml"
export VLLM_USE_LIGHTOP_MOE_SUM_MUL_ADD=1
export USE_FUSED_RMS_QUANT=1
export DEBUG_CLR_GRAPH_PACKET_CAPTURE=false
export VLLM_SPEC_DECODE_EAGER=1
#----pp 并行开启如下变量----
export VLLM_SCHED_ENABLE_MINIMAL_INJECTION=1
#----如遇到运行过程中的 OOM 增加下面设置----
export VLLM_USE_GLOBAL_CACHE13=1
export VLLM_FUSED_MOE_CHUNK_SIZE=16384
export VLLM_USE_OPT_OP=1
export VLLM_ZERO_OVERHEAD=0
export VLLM_ENABLE_TBO=0
export VLLM_PP_LAYER_PARTITION=35,26
export Allgather_Base_STREAM_WITH_COMPUTE=1
export SENDRECV_STREAM_WITH_COMPUTE=1
export VLLM_USE_CUDA_GRAPH_SIZES=1 #(默认)
export VLLM_USE_LIGHTOP_FILL_MOE_ALIGN=1
export VLLM_USE_OPT_ZEROS=1
export VLLM_USE_PP_SYNC=1
export USE_FUSED_SILU_MUL_QUANT=1
export VLLM_REJECT_SAMPLE_OPT=1  #新增

export VLLM_ZERO_OVERHEAD_ENHANCE=1  #新增
```

7.4. 启动 head ray

注意 IP 要用内网地址，可参考，注意修改您的 IP： 10.206.120.2

```
ray start --head --node-ip-address=10.206.120.2  --port=1222 --num-gpus=8 --num-cpus=32
```

```
[root@ip-10-206-120-2 ~]# ifconfig |grep -i eth0 -A 9
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.206.120.2  netmask 255.255.255.0  broadcast 10.206.120.255
        inet6 fe80::5054:ff:fead:e0c9  prefixlen 64  scopeid 0x20<link>
        ether 52:54:00:ad:e0:c9  txqueuelen 1000  (Ethernet)
        RX packets 2956382438  bytes 4235012890219 (3.8 TiB)
```

7.5. 启动 node ray

在第二台机器上启动 node ray 增加 head，组成双机；注意这里的--address 使用 head 节点的内网 IP 如上一小节。

```
ray start --address='10.206.120.2:1222' --num-gpus=8 --num-cpus=32
```

7.6. 检查 ray 是否成功

在上述双节点上任何一个节点上，进入容器，可以检查如下：看到 2 个 node 表示 ray 组建成功。

```
ray status
```

```
root@VM-120-46-tencentos:/workspace# ray status
======== Autoscaler status: 2025-12-05 20:30:17.928122 ========
Node status
---------------------------------------------------------------
Active:
 1 node_232926f602c6ce45e43c1b55bca6e9ef8854b4b099e9a70884cd354e
 1 node_ab8c7f36b3b6cfddd81a5f362e6c6578fd471a28a478039c5cd864ab
Pending:
 (no pending nodes)
Recent failures:
 (no failures)

Resources
---------------------------------------------------------------
Total Usage:
 0.0/64.0 CPU
 0.0/16.0 GPU
 0B/3.72TiB memory
 0B/372.53GiB object_store_memory
```

7.7. server 命令

在 head node 上容器里面执行:

```
vllm serve /data0/model/DeepSeek-R1-0528-Channel-INT8/ --host 0.0.0.0 --port 8008 \
    --dtype bfloat16 \
     -pp 2 -tp 8 \
    --trust-remote-code \
    --distributed-executor-backend ray \
    --max-model-len 32768 --max-seq-len-to-capture 8192 \
    --gpu-memory-utilization 0.95 --max-num-seqs 512 \
    --disable-log-requests --block-size 64 --enable-chunked-prefill --max-num-batched-tokens 8192 \
    --enable-prefix-caching \
    --kv-cache-dtype fp8_e5m2 -q slimquant_marlin \
    --compilation-config '{"full_cuda_graph": true}' \
    --speculative_config '{"method": "deepseek_mtp", "num_speculative_tokens": 3}' \
    2>&1 | tee  deepseek_log.log
```

注意: 目前海光 DS 推理性能最好的是 pp2tp8 方案

7.8. client 命令示例

示例测试脚本：（这里给了一个示例，客户可以根据自己需要修改输入输出等场景）

```
model_path=/data0/model/DeepSeek-R1-0528-Channel-INT8/
model=${model_path##*/}
tp=16
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./client/${model}-${mode}-$time-test"
#logdir="${logpath}/${model}-tp${tp}-${time}-${num_prompts}-in${input}-
out${output}.log"
#logdir=${logpath}/${model}-tp${tp}-${time}-${batch}-in${prompt_tokens}-
out${completion_tokens}.log
data_type="fp16"
port=8008
csv_name="${model}-tp${tp}-$time"

if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi
all_log="${logpath}/all.csv"



echo
"input,output,request_rate,num_prompts,concurrency,duration_s,rps,generate_throughput_t
ok_s,total_throughput_tok_s,mean_ttft_ms,p95_ttft_ms,p99_ttft_ms,mean_tpot_ms,p95_tpot_
ms,p99_tpot_ms,mean_itl_ms,p95_itl_ms,p99_itl_ms" > $all_log

pairs=(
  "1005 635" "200 1024"
)

#--profile
for pair in "${pairs[@]}"; do
  prompt_tokens=$(echo "$pair" | awk '{print $1}')
  completion_tokens=$(echo "$pair" | awk '{print $2}')
for batch in 256 32 64 128; do
#for batch in 256; do
      concurrency_multiplier=8
      if [ $batch -gt 128 ]; then
         concurrency_multiplier=4
      fi
      vllm bench serve \
      --model ${model_path} \
```

7.9. 结果参考

参考测试结果

卡型	测试配置	测试参数		评估指标											
				TTFT			TPOT			ITL			Request throughput	Total Token throughput	Su
		输入/输出	并发	mean	P99	vs H20	mean	P99	vs H20	mean	P99	vs H20			
海光BW1000	两机16卡/int8精度 tp8+pp2	1024/1024	1	203.76	212.7	1.06	12.41	15.51	0.43	34.57	35.73	0.51	0.08	158.67	
			10	271.24	659.29	1.59	17.58	25.32	0.83	59.53	258.21	0.77	0.53	1081.52	
			20	261.63	590.1	1.59	23.52	32.19	0.99	80.43	278.68	0.89	0.8	1629.93	
			50	328.53	946.48	1.41	37.4	54.29	0.77	127.78	386.59	0.70	1.26	2569.77	
			100	453.91	2061.58	2.40	51.64	73.42	0.77	180.67	565.14	0.66	1.81	3696.59	
			128	863.58	7432.82	1.55	59.77	81.16	0.72	207.72	869.81	0.63	2	4086.88	

7.10. 参考

上面章节，我们是在海光的 Deepseek 推理实施的文档上构建更适配客户 PoC 的文档。

客户如果有需要也可以直接从海光的容器镜像上开始构建 deepseek 的推理环境。

截止：20251203 给的推荐安装方式。

```
# 文档里需要的物料可下面地址下载
wget -c https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/external/haiguang/inference/Bw1000-llm-inference-20251212.zip
```

8. Qwen 推理测试

8.1. 权重下载

这个章节可以支持类似模型如下：(Qwen3-32B/Qwen2.5-72B-Instruct/Qwen2.5-VL-72B-Instruct/Qwen2.5-VL-8B//Qwen2.5-VL-72B-Instruct-AWQ)

Qwen 模型权重如果原生是 bf16，则 BW1000 直接支持。如果是 FP8，则同样需要权重转换。

Qwen 模型的下载，是公网的可以参考如下网站（任一即可）

通义千问 2.5-72B-预训练 模型下载部分

https://huggingface.co/Qwen/Qwen3-32B

8.2. docker 镜像及容器

镜像/容器可以和 DeepSeek 同一个。

```
docker pull ccr.ccs.tencentyun.com/taco/taco-infer:HG-BW1000-vllm-0.9.2-ubuntu22.04-das.opt5.dtk25044.20260128.gf35ea024-py3.10-20260202
```

启动容器：镜像可根据测试需求选择

```
docker run -dit \
  --network=host \
  --name=xxxxxx \
  --privileged \
  --device=/dev/kfd \
  --device=/dev/dri \
  --ipc=host \
  --shm-size=128G  \
  --group-add video \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  -u root \
  --ulimit stack=-1:-1 \
  --ulimit memlock=-1:-1 \
  -v /opt/hyhal:/opt/hyhal:ro \
  -v /data0:/data0  \
  -v /workspace:/root  \
ccr.ccs.tencentyun.com/taco/taco-infer:HG-BW1000-vllm-0.9.2-ubuntu22.04-das.opt5.dtk25044.20260128.gf35ea024-py3.10-20260202
```

8.3. server 命令

在 head node 上容器里面执行：

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export HIP_VISIBLE_DEVICES=4,5,6,7
export HSA_FORCE_FINE_GRAIN_PCIE=1
export NCCL_MAX_NCHANNELS=16
export NCCL_MIN_NCHANNELS=16
export NCCL_P2P_LEVEL=SYS
export NCCL_LAUNCH_MODE=GROUP
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=0
export VLLM_RANK1_NUMA=0
export VLLM_RANK2_NUMA=0
export VLLM_RANK3_NUMA=0
export VLLM_SPEC_DECODE_EAGER=1
export VLLM_ZERO_OVERHEAD=1 # zero

model_path=/model/Qwen3-32B
model=${model_path##*/}
tp=4
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./logs-server/"
data_type="fp16"
port=8089
if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi

vllm serve $model_path \
      --host 0.0.0.0 \
      --port $port   \
      --dtype float16 \
      --tensor-parallel-size $tp   \
      --trust-remote-code  \
      --max-num-seqs 1024   \
      --max-seq-len-to-capture 40960    \
      --distributed-executor-backend=mp \
      --enable-prefix-caching  \
```

8.4. client 命令示例

示例测试脚本：（这里给了一个示例，客户可以根据自己需要修改输入输出等场景）

```
model_path=/model/Qwen3-32B
model=${model_path##*/}
tp=4
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./client/${model}-${mode}-$time-test"
#logdir="${logpath}/${model}-tp${tp}-${time}-${num_prompts}-in${input}-
out${output}.log"
#logdir=${logpath}/${model}-tp${tp}-${time}-${batch}-in${prompt_tokens}-
out${completion_tokens}.log
data_type="fp16"
port=8089
csv_name="${model}-tp${tp}-$time"

if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi
all_log="${logpath}/all.csv"



echo
"input,output,request_rate,num_prompts,concurrency,duration_s,rps,generate_throughput_t
ok_s,total_throughput_tok_s,mean_ttft_ms,p95_ttft_ms,p99_ttft_ms,mean_tpot_ms,p95_tpot_
ms,p99_tpot_ms,mean_itl_ms,p95_itl_ms,p99_itl_ms" > $all_log

pairs=(
  "4096 1024"
  "2048 1024"
  "8096 2048"
)

#--profile
for pair in "${pairs[@]}"; do
  prompt_tokens=$(echo "$pair" | awk '{print $1}')
  completion_tokens=$(echo "$pair" | awk '{print $2}')
for batch in 1 8 16; do
#for batch in 256; do
      concurrency_multiplier=8
      if [ $batch -gt 128 ]; then
         concurrency_multiplier=4
      fi
```

8.5. 多模态 server 命令

8.5.1. 镜像下载

```
docker pull ccr.ccs.tencentyun.com/taco/infer-poc:vllm0.9.2-ubuntu22.04-dtk26.04-0130-py3.10-20260204-qwen-vl
```

8.5.2. Qwen2.5-VL-72B-Instruct：

在容器里面执行:

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export HIP_VISIBLE_DEVICES=4,5,6,7
export HSA_FORCE_FINE_GRAIN_PCIE=1
export NCCL_MAX_NCHANNELS=16
export NCCL_MIN_NCHANNELS=16
export NCCL_P2P_LEVEL=SYS
export NCCL_LAUNCH_MODE=GROUP
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=0
export VLLM_RANK1_NUMA=0
export VLLM_RANK2_NUMA=0
export VLLM_RANK3_NUMA=0
export VLLM_SPEC_DECODE_EAGER=1
export VLLM_ZERO_OVERHEAD=1 # zero


model_path=/data0/model/Qwen/Qwen2.5-VL-72B-Instruct
model=${model_path##*/}
tp=4
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./logs-server/"
data_type="fp16"
port=8089
if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi

#--max-num-seqs 1024
#--enforce-eager

vllm serve $model_path \
      --host 0.0.0.0 \
      --port $port   \
      --dtype float16 \
      --tensor-parallel-size $tp   \
      --trust-remote-code \
```

8.5.3. Qwen2.5-VL-3B-Instruct

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export HIP_VISIBLE_DEVICES=3
export HSA_FORCE_FINE_GRAIN_PCIE=1
export NCCL_MAX_NCHANNELS=16
export NCCL_MIN_NCHANNELS=16
export NCCL_P2P_LEVEL=SYS
export NCCL_LAUNCH_MODE=GROUP
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=1
export VLLM_SPEC_DECODE_EAGER=1
export VLLM_ZERO_OVERHEAD=1 # zero
#export TORCH_COMPILE_DEBUG=1

#export VLLM_USE_FUSED_RMS_ROPE=1
#export VLLM_USE_OPT_RESHAPE_AND_CACHE=1

model_path=/data0/model/Qwen/Qwen2.5-VL-3B-Instruct
model=${model_path##*/}
tp=1
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./logs-server/"
data_type="fp16"
port=19800
if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi

#--max-num-seqs 1024
#--enforce-eager

vllm serve $model_path \
      --host 0.0.0.0 \
      --port $port   \
      --dtype float16 \
      --tensor-parallel-size $tp   \
      --trust-remote-code \
```

8.6. 多模态 client 示例

8.6.1. Qwen2.5-VL-72B-Instruct

示例测试脚本：（这里给了一个示例，客户可以根据自己需要修改输入输出等场景）

数据集可以参考下载：

```
wget -c https://taco-public-1251001002.cos.ap-shanghai.myqcloud.com/datasets/qwen2.5vl/buss_mmec_id_48_model_qwen2.5-vl-72b-instruct.json
```

client 参数：

```
model_path=/data0/model/Qwen/Qwen2.5-VL-72B-Instruct
model=${model_path##*/}
tp=4
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./client/${model}-${mode}-$time-test"
#logdir="${logpath}/${model}-tp${tp}-${time}-${num_prompts}-in${input}-
out${output}.log"
#logdir=${logpath}/${model}-tp${tp}-${time}-${batch}-in${prompt_tokens}-
out${completion_tokens}.log
data_type="fp16"
port=8089
csv_name="${model}-tp${tp}-$time"

if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi
all_log="${logpath}/all.csv"



echo
"input,output,request_rate,num_prompts,concurrency,duration_s,rps,generate_throughput_t
ok_s,total_throughput_tok_s,mean_ttft_ms,p95_ttft_ms,p99_ttft_ms,mean_tpot_ms,p95_tpot_
ms,p99_tpot_ms,mean_itl_ms,p95_itl_ms,p99_itl_ms" > $all_log

pairs=(
  "1024 1024"
)
#--profile
for pair in "${pairs[@]}"; do
  prompt_tokens=$(echo "$pair" | awk '{print $1}')
  completion_tokens=$(echo "$pair" | awk '{print $2}')
for batch in 1 16 32 64 128 256; do
#for batch in 256; do
      concurrency_multiplier=8
      if [ $batch -gt 128 ]; then
         concurrency_multiplier=4
      fi
      vllm bench serve \
      --model ${model_path} \
      --dataset-path /data0/model/dataset/buss_mmec_id_48_model_qwen2.5-vl-72b-
```

8.6.2. Qwen2.5-VL-3B-Instruct

```
wget https://huggingface.co/datasets/Lin-Chen/ShareGPT4V/resolve/main/sharegpt4v_instruct_gpt4-vision_cap100k.json
```

```
#!/bin/bash

# 配置参数
model_path="/data0/model/Qwen/Qwen2.5-VL-3B-Instruct"
model="${model_path##*/}"
time=$(date "+%m%d-%H%M")
logpath="./client/${model}-${time}-test"

# 创建日志目录
if [ ! -d ${logpath} ]; then
   mkdir -p ${logpath}
fi

# 定义 CSV 文件
CSV_FILE="${logpath}/performance_stats.csv"

# 初始化 CSV 文件表头
echo
"test_time,concurrency,mean_ttft_ms,p95_ttft_ms,p99_ttft_ms,mean_tpot_ms,p95_tpot_ms,p99_tpot_ms,mean_itl_ms,p95_itl_ms,p99_itl_ms,total_throughput_tok_s,output_throughput_tok_s,total_input_tokens,total_output_tokens,benchmark_duration_s,request_throughput_rps" >
$CSV_FILE

# 定义并发数数组
concurrency_list=(8 32 64 128 256)

echo "开始多并发测试: $(date)"
echo "模型: $model"
echo "并发测试列表: ${concurrency_list[@]}"
echo "==========================================="

for concurrency in "${concurrency_list[@]}"; do
   echo "正在测试并发数: $concurrency"

   # 为每个并发测试创建单独的日志文件
   LOG_FILE="${logpath}/test_${time}_concurrency_${concurrency}.log"
   TEMP_LOG="${logpath}/temp_${time}_concurrency_${concurrency}.log"

   # 运行性能测试
   vllm bench serve \
```

8.7. AWQ Server

Qwen2.5-VL-72B-Instruct-AWQ 推理：使用最新镜像：

```
ccr.ccs.tencentyun.com/taco/infer-poc:vllm-0.11.0-ubuntu22.04-dtk26.04-0130-py3.10-20260204-awq
```

server 端：

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export HIP_VISIBLE_DEVICES=0,1
export HSA_FORCE_FINE_GRAIN_PCIE=1
export NCCL_MAX_NCHANNELS=16
export NCCL_MIN_NCHANNELS=16
export NCCL_P2P_LEVEL=SYS
export NCCL_LAUNCH_MODE=GROUP
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=0
export VLLM_RANK1_NUMA=1
export VLLM_SPEC_DECODE_EAGER=1
export VLLM_ZERO_OVERHEAD=1 # zero

export VLLM_USE_LIGHTOP_RMS_ROPE_CONCAT=0

log_date=$(date "+%Y-%m-%d")
log_dir="./${log_date}"
if ! mkdir -p "${log_dir}"; then
   exit 1
fi
model_path=/data0/model/Qwen2.5-VL-72B-Instruct-AWQ
model=${model_path##*/}
time=$(date "+%Y-%m-%d-%H-%M-%S")
data_type="float16"
tp=2
port=8023
gpu_memory=0.92

AWQ_GEMM_MARLIN=1  VLLM_USE_TRITON_AWQ=0 vllm serve ${model_path} \
      --host 0.0.0.0 --port ${port} \
      --dtype float16 \
      -tp ${tp} \
      --trust-remote-code \
      --max-model-len 8192 \
      --gpu-memory-utilization ${gpu_memory} \
      --max-num-seqs 512 \
      --disable-log-requests \
      --disable-cascade-attn \
```

8.8. AWQ client 示例

示例测试脚本：（这里给了一个示例，客户可以根据自己需要修改输入输出等场景）

当前关注：功能跑通。

client 参数：

```
#!/usr/bin/env bash
set -euo pipefail

rm -rf ~/.cache
model_path=/data0/model/Qwen2.5-VL-72B-Instruct-AWQ
model=${model_path##*/}
tp=2
port=8023
dtype="float16"
time=$(date "+%m%d-%H%M")
modeType="cudagraph"
current_time=$(date +"%Y%m%d-%H%M")
logpath="./${model}/${modeType}/${current_time}"

if [ ! -d "${logpath}" ]; then
  mkdir -p ${logpath}
fi

generate_batches() {
  local prompt_tokens=$1
  local completion_tokens=$2
  local batches=()

  case "${prompt_tokens},${completion_tokens}" in
    "2000,3200")
      for ((i=1; i<=32; i*=2)); do
        batches+=($i)
      done
      ;;
    "2000,5800")
      for ((i=1; i<=32; i*=2)); do
        batches+=($i)
      done
      ;;
  esac

  echo "${batches[@]}"
}

#pairs=("1005 635" "200 1024")
#pairs=("1005 635")
#pairs=("200 1024")
```

本例输出结果：

```
root@VM-120-46-tencentos:/mnt/why/qwen-72b-vl-awq# bash test.py
Question: 总结一段关于商业的内容和小红帽大灰狼的故事，和42号混凝土的故事，需要包括国产DCU。共计100字

=== Base (/model/Qwen2.5-VL-72B-Instruct-AWQ) ===

商业世界中，国产DCU正引领科技革新，推动产业升级。小红帽与大灰狼的故事教会我们识别风险，谨慎前行。而42号混凝土，以其卓越性能，成为建筑行业的基石。三者结合，寓意在复杂多变的市场环境中，需具备敏锐洞察力、稳健基础及创新技术，方能立于不败之地。

=== LoRA1 (lora1) ===
商业中，国产DCU在市场中崭露头角，助力企业创新。小红帽与大灰狼的故事教会我们警惕与智慧。42号混凝土的诞生，象征着技术进步与工程奇迹。三者交织，展现人类智慧与勇气，推动社会向前发展。

=== LoRA2 (lora2) ===
商业中，国产DCU在市场中崭露头角，助力企业智能化升级。小红帽与大灰狼的故事教会我们警惕与智慧。而42号混凝土，以其卓越的性能，在建筑行业广泛应用，成为现代工程的基石。三者虽领域不同，却都体现了创新与发展的精神。
```

8.9. AWQ 精度验证

待提供。预计明天完成。

8.10. AWQ 性能测评

并发性能测试：可以从这里拉测试的脚本性能说明

> https://haihub-model-nj-1251001002.cos.ap-nanjing.myqcloud.com/customers/hqzn/qwen-72b-vl-awq.tar.gz

8.11. 结果参考

参考测试结果如下图；

input	output	request_rate	num_prompts	concurrency	duration_s	rps	generate_throug	total_throughpu	mean_ttft_ms	p95_ttft_ms	p99_ttft_ms	mean_tpot_ms	p95_tpot_ms	p99_tpot_ms	mean_itl_ms	p95_itl_ms	p99_itl_ms	vs total-throughp	vs mean_tpot_ms	vs mean_ttft_ms
2000	3200	inf	1	1	106.76	0.01	29.97	48.71	124.68	124.68	124.68	33.33	33.33	33.33	33.32	33.48	33.74	1.147467609	1.145214521	3.14894129
2000	3200	inf	2	2	111	0.02	57.66	93.7	1340.77	1379.71	1383.17	34.28	34.29	34.29	34.27	34.72	34.87	3.80275974	3.805134189	3.510087487
2000	3200	inf	4	4	119.1	0.03	107.47	174.23	1651.37	2781.77	2928.24	36.71	37.11	37.17	36.7	38.35	42.02	1.187419069	1.147643694	4.025536373
2000	3200	inf	8	8	136.93	0.06	186.96	303.46	3776.7	5767.49	6063.82	41.61	42.56	42.7	41.6	43.23	43.79	1.154630546	1.147320356	1.397060926
2000	3200	inf	16	16	176.54	0.09	290.02	471.01	7058.71	11820.38	11825.21	52.96	54.1	54.76	52.95	55.23	55.68	1.191193951	1.181268882	1.412610803
2000	3200	inf	32	32	328.29	0.1	311.91	506.71	13911.85	24179.39	24191.61	98.26	101.81	102.27	98.23	105	106.35	3.618840166	3.726643599	1.153219737
2000	5800	inf	1	1	167.69	0.01	29.62	41.55	120.04	120.04	120.04	33.74	33.74	33.74	33.74	34.57	35.71	1.198788229	1.148192057	3.243168944
2000	5800	inf	2	2	172.98	0.01	57.43	80.55	430.91	699.09	722.93	34.74	34.8	34.8	34.74	35.6	37.07	3.954344624	3.777777778	10.9123947
2000	5800	inf	4	4	186.83	0.02	106.6	149.16	1491.13	1769.31	1769.75	36.98	37.12	37.15	36.97	38.57	40.05	1.212978775	1.166576528	2.102532978
2000	5800	inf	8	8	215.53	0.04	184.59	258.6	3158.46	3515.06	3516.15	42.42	42.73	42.89	42.41	45.52	48.51	1.242492673	1.187411598	2.33398555
2000	5800	inf	16	16	297.06	0.05	267.69	375.25	9573.68	21948.47	21953.09	57.54	58.89	59.32	57.53	61.52	62.58	1.133994137	1.091414668	1.325493436
2000	5800	inf	32	32	564.58	0.06	281.61	394.89	12276.93	23150.48	23172.21	101.03	106.88	109.09	101.01	104.67	106.01	3.821639408	3.866277343	1.502806483
																	优化提升	2.055545744	2.032572934	3.005653226

如需要更多的测试结果文档，可以咨询腾讯侧提供相关文档。

8.12. 双机 Qwen3-235B-A22B-Instruct-2507

8.12.1. docker 镜像及容器

qwen3-235B 需要两台机器，要在两台机器分别下载镜像

```
docker pull ccr.ccs.tencentyun.com/taco/infer-poc:vllm0.15.1-ubuntu22.04-dtk26.04-0130-py3.10-qwen3-235B
```

两台容器分别启动容器

```
docker run -dit \
  --network=host \
  --name=qwen-235B \
  --privileged \
  --device=/dev/kfd \
  --device=/dev/dri \
  --ipc=host \
  --shm-size=128G  \
  --group-add video \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  -u root \
  --ulimit stack=-1:-1 \
  --ulimit memlock=-1:-1 \
  -v /opt/hyhal:/opt/hyhal:ro \
  -v /data1:/data1  \
  -v /workspace:/root  \
ccr.ccs.tencentyun.com/taco/infer-poc:vllm0.15.1-ubuntu22.04-dtk26.04-0130-py3.10-qwen3-235B
```

8.12.2. server 命令

设置环境变量：把下面代码放在 env.sh（自己新建）， 然后执行：source env.sh
容器内/workspace/env.sh：

```
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_HOST_IP=$(hostname -I | awk '{print $1}')
echo $VLLM_HOST_IP
export NCCL_SOCKET_IFNAME=eth0
export GLOO_SOCKET_IFNAME=eth0
export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
#export
NCCL_IB_HCA=mlx5_0:1,lx5_2:1,mlx5_3:1,mlx5_4:1,mlx5_5:1,mlx5_6:1,mlx5_7:1,mlx5_8:1,m
lx5_9:1
#export  NCCL_TOPO_FILE="/topo-input.xml"
#export
NCCL_IB_HCA=mlx5_bond_0:1,mlx5_bond_1:1,mlx5_bond_2:1,mlx5_bond_3:1,mlx5_bond_4:
1,mlx5_bond_5:1,mlx5_bond_6:1,mlx5_bond_7:1
export NCCL_MIN_NCHANNELS=32
export NCCL_MAX_NCHANNELS=32
export NCCL_NET_GDR_READ=1
export VLLM_RPC_TIMEOUT=1800000
#export VLLM_USE_GLOBAL_CACHE13=1
export RAY_CGRAPH_get_timeout=300
#export NCCL_TOPO_MAPPING_FILE="/topo_mapping.xml"


export VLLM_USE_LIGHTOP=1
export PYTORCH_MIOPEN_SUGGEST_NDHWC=0
export HIP_USE_GRAPH_QUEUE_POOL=1
#export NCCL_NCHANNELS_PER_PEER=2
export VLLM_USE_FUSED_RMS_ROPE=0


export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=1
export VLLM_RANK1_NUMA=1
export VLLM_RANK2_NUMA=1
export VLLM_RANK3_NUMA=1
export VLLM_RANK4_NUMA=0
export VLLM_RANK5_NUMA=0
export VLLM_RANK6_NUMA=0
export VLLM_RANK7_NUMA=0

export NCCL_SOCKET_IFNAME=eth0
export NCCL_IB_GID_INDEX=3
export NCCL_IB_DISABLE=0
```

设置 ray-cli 集群,--node-ip-address 要设置成主节点 ip

```
# 先在一台机器执行这个命令做主节点
ray start --head --node-ip-address=10.206.120.2 --port=6569 --num-gpus=8 --num-cpus=16

# 另外一台机器启动下面这个命令
ray start --address='10.206.120.2:6569' --num-gpus=8 --num-cpus=16
```

在主节点启动 vllm 服务端，注意模型目录，容器内/workspace/start.sh:

```
#!bin/sh
#export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
#export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
#rm -rf ~/.cache/
#rm -rf ~/.triton/

#export VLLM_USE_LIGHTOP=1
# 0306 2328 注掉下面这行
#export ALLREDUCE_STREAM_WITH_COMPUTE=1

#export VLLM_HOST_IP=$(hostname -I | awk '{print $1}')
echo $VLLM_HOST_IP

model_path=/data1/models/Qwen3-VL-235B-A22B-Instruct
model=Qwen3-VL-235B-A22B-Instruct
logpath="./logs/"
time=$(date "+%m%d-%H%M")
port=9501

#if [ "$RANK" = "0" ]; then
  # 启动 Ray head
  #ray start --head --port 6700 --num-gpus=8
  # 启动 vLLM 服务（这会阻塞）
  vllm serve "$model_path" \
    --served-model-name "${model}" \
    --host 0.0.0.0 \
    --port "$port" \
    --dtype float16 \
    -tp 16 \
    --trust-remote-code \
    --distributed-executor-backend ray \
    --max-model-len 65536 \
    --gpu-memory-utilization 0.8 \
    --max-num-seqs 65 \
    --enable-chunked-prefill \
    --enable-prefix-caching

    #--kv-cache-dtype fp8_e5m2 \
    #--compilation-config '{"full_cuda_graph": true}' \
#else
  # 启动 Ray worker
```

8.12.3. client 命令示例

再开一个容器窗口，运行服务端，注意模型路径和 --dataset-path 参数的目录位于容器内/workspace/Evaltest/vl_test_gpu.py：

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
批量性能压测脚本（按分辨率 + 多组 number/parallel） + NPU/内存资源监控
- 命令行参数: <resolution>
- 监控数据会自动保存在每次压测的输出目录下的 monitor_data.csv 中
"""

import my_multimodal_dataset_7b  # noqa
import my_multimodal_dataset_7b_4w  # noqa
import bailian_vl_api_7b as bailian_vl_api       # noqa
from evalscope.perf.arguments import Arguments
from evalscope.perf.main import run_perf_benchmark
import os
import sys
import time
import subprocess
import re
import csv
import threading
from datetime import datetime

# ================== 配置区 ==================
base_dataset_dir = "datasets/image_under"
model = "Qwen3-VL-235B-A22B-Instruct"
#model = "/data1/models/Qwen2.5-VL-7B-Instruct"
api = "bailian_vl"
#url = "http://192.168.0.137:9005/v1/chat/completions"
url = "http://127.0.0.1:9501/v1/chat/completions"
max_tokens = 10000
temperature = 0.7
debug = True
stream = True

# 监控采样间隔 (秒)
MONITOR_INTERVAL = 5



# ============================================

#
```

8.13. Qwen3.5-397B-int8

```
docker pull ccr.ccs.tencentyun.com/taco/infer-poc:vllm0.15.1-ubuntu22.04-dtk26.04-0130-py3.10-qwen3.5-397B
```

serve:

```
rm -rf ~/.cache/
rm -rf ~/.triton/

export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export VLLM_NUMA_BIND=1
export VLLM_RANK0_NUMA=3
export VLLM_RANK1_NUMA=1
export VLLM_RANK2_NUMA=1
export VLLM_RANK3_NUMA=0
export VLLM_RANK4_NUMA=7
export VLLM_RANK5_NUMA=5
export VLLM_RANK6_NUMA=5
export VLLM_RANK7_NUMA=4

export USE_FUSED_SILU_MUL_QUANT=1
export VLLM_V1_USE_FA_UNIFIED_ATTN_2D=1
export VLLM_USE_FUSED_RMS_ROPE=1
export VLLM_USE_OPT_RESHAPE_AND_CACHE=1

export VLLM_USE_TOPK_RENORM=1 #lightop topk_softmax
export LMSLIM_USE_LIGHTOP=1
export VLLM_USE_LIGHTOP_MOE_ALIGN=1
export VLLM_USE_LIGHTOP_FILL_MOE_ALIGN=1
export VLLM_USE_LIGHTOP_FUSED_TOPP_TOPK=1

vllm serve  /public/opendas/DL_DATA/llm-models/qwen3.5/Qwen3.5-397B-A17B-W8A8/
--host 0.0.0.0 --port 8008 -tp 8  --trust-remote-code \
   --max-model-len 150000 --gpu-memory-utilization 0.9 --max-num-seqs 512 \
   --disable-log-requests --block-size 64 --enable-chunked-prefill --max-num-batched-tokens
8192 \
   --no-enable-prefix-caching --reasoning-parser qwen3 -q slimquant_marlin
```

client:

```
model_path=/public/opendas/DL_DATA/llm-models/qwen3.5/Qwen3.5-397B-A17B-W8A8/
model=${model_path##*/}
tp=8
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
logpath="./client/${model}-${mode}-$time-test"
#logdir="${logpath}/${model}-tp${tp}-${time}-${num_prompts}-in${input}-out${output}.log"
#logdir=${logpath}/${model}-tp${tp}-${time}-${batch}-in${prompt_tokens}-out${completion_tokens}.log
data_type="fp16"
port=8008
csv_name="${model}-tp${tp}-$time"

if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi
all_log="${logpath}/all.csv"

echo "input,output,request_rate,num_prompts,concurrency,duration_s,rps,generate_throughput_tok_s,total_throughput_tok_s,mean_ttft_ms,p95_ttft_ms,p99_ttft_ms,mean_tpot_ms,p95_tpot_ms,p99_tpot_ms,mean_itl_ms,p95_itl_ms,p99_itl_ms" > $all_log

pairs=(
  "5000 5000"
  "128000 16000"
)
# pairs=(
#   "128000 16000"
# )

#--profile
for pair in "${pairs[@]}"; do
  prompt_tokens=$(echo "$pair" | awk '{print $1}')
  completion_tokens=$(echo "$pair" | awk '{print $2}')
for batch in 1 4 8 16 ; do
#for batch in 256; do
      concurrency_multiplier=8
      if [ $batch -gt 128 ]; then
```

8.14. Kimi2.5-int4

镜像：

```
docker pull ccr.ccs.tencentyun.com/taco/taco-infer:HG-BW1000-0.15.1-das.opt1.alpha.dtk2604.torch290.2604081608.g1b8e8d-ubuntu22.04-py3.10-20260424
```

环境：

```
set -euo pipefail
export VLLM_HOST_IP=$(hostname -I | awk '{print $1}')
echo "[INFO] VLLM_HOST_IP=${VLLM_HOST_IP}"
echo $VLLM_HOST_IP
export GLOO_SOCKET_IFNAME=ib0
export NCCL_SOCKET_IFNAME=ib0
export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export NCCL_IB_HCA=shca_0:1,shca_1:1,shca_2:1,shca_3:1
export HSA_FORCE_FINE_GRAIN_PCIE=1

export NCCL_MAX_NCHANNELS=16
export NCCL_MIN_NCHANNELS=16
export NCCL_P2P_LEVEL=SYS
export NCCL_LAUNCH_MODE=GROUP
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_USE_V1=1

export VLLM_TORCH_PROFILER_DIR=/mnt/kimi-2.5/prof/
export VLLM_USE_LIGHTOP=1
export VLLM_USE_PIECEWISE=0
export VLLM_USE_LIGHTOP_FILL_MOE_ALIGN=1
export VLLM_1D_MROPE=1
export USE_FUSED_SILU_MUL_QUANT=0
export USE_FUSED_RMS_QUANT=0
export VLLM_USE_LIGHTOP_RMS_ROPE_CONCAT=1
export VLLM_USE_FUSED_FILL_RMS_CAT=0
export VLLM_USE_LIGHTOP_FUSED_TOPP_TOPK=1
export VLLM_USE_FUSED_RMS_ROPE=1
export VLLM_W8A8_BACKEND=3
export MOE_NN=0
export VLLM_V1_USE_FUSED_QKV_SPLIT_RMS_ROPE_KVSTORE=0
export VLLM_USE_FLASH_ATTN_FP8=1
export LD_LIBRARY_PATH=/opt/rocblas-install/lib:$LD_LIBRARY_PATH

export VLLM_ROCM_USE_AITER_MOE=1
```

node1:

```
rm -rf ~/.cache/
rm -rf ~/.triton/

model_path=/module/Kimi-K2.5
model=${model_path##*/}
tp=16
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
port=8032
logpath="./server/${model}-tp${tp}-$time"
data_type="bfloat16"
if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi

vllm serve $model_path \
   --port $port \
   --max-model-len 81920 \
   --dtype $data_type \
   -pp 2 -tp 8 \
   --trust-remote-code \
   --nnode 2 \
   --node-rank 0 --master-addr 10.16.1.78 \
   --gpu-memory-utilization 0.9 \
   --enable-prefix-caching \
   2>&1 | tee  ${logpath}-$time.log
```

node2:

```
rm -rf ~/.cache/
rm -rf ~/.triton/

model_path=/module/Kimi-K2.5
model=${model_path##*/}
tp=16
time=$(date "+%m%d-%H%M")
#mode="eager"
mode="cudagraph"
port=8032
logpath="./server/${model}-tp${tp}-$time"
data_type="bfloat16"
if [ ! -f ${logpath} ]; then
   mkdir ${logpath} -p
fi

vllm serve $model_path \
   --max-model-len 81920 \
   --dtype $data_type \
   -pp 2 -tp 8 \
   --trust-remote-code \
   --nnode 2 \
   --node-rank 1 --master-addr 10.16.1.78 \
   --gpu-memory-utilization 0.9 \
   --enable-prefix-caching \
   --headless \
   2>&1 | tee  ${logpath}-$time.log
```

8.15. GLM5-int8

镜像:

```

```

环境:

```
rm -rf ~/.cache
rm -rf ~/.triton
export VLLM_HOST_IP=$(hostname -I | awk '{print $1}')
export NCCL_SOCKET_IFNAME=ib0
export GLOO_SOCKET_IFNAME=ib0
export VLLM_TORCH_PROFILER_DIR=/prof
export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export ALLREDUCE_STREAM_WITH_COMPUTE=1
export NCCL_MIN_NCHANNELS=16
export NCCL_MAX_NCHANNELS=16
export VLLM_USE_PP_BALANCE=1
export NCCL_IB_HCA=^mlx5_0

export Allgather_Base_STREAM_WITH_COMPUTE=1
export SENDRECV_STREAM_WITH_COMPUTE=1
export HIP_KERNEL_EVENT_SYSTENFENCE=1
export VLLM_RPC_TIMEOUT=1800000
export VLLM_USE_PIECEWISE=1 # 需暂时打开 piecewise，fullgraph 有精度问题
export VLLM_REJECT_SAMPLE_OPT=1 # 宽松采样，提高 mtp 接受率从而提高 tpot 性能，略微影响精度 default 1
#===============融合算子=================
export USE_FUSED_RMS_QUANT=0 #default 0
export USE_FUSED_SILU_MUL_QUANT=1 #default 0
#===============共享专家融合 已知精度异常=============
export VLLM_ROCM_USE_AITER=0 #default 0
export VLLM_ROCM_USE_AITER_MOE=0 #default 0
export VLLM_ROCM_USE_AITER_FUSION_SHARED_EXPERTS=0 #default 0
#注意：--compilation-config 中需要添加"custom_ops": ["all", "+rms_norm"]
export VLLM_SPEC_DECODE_EAGER=0 #MTP 使用 eager 下发 default 0
export VLLM_USE_GLOBAL_CACHE13=1 #减少显存碎片化 default 0
export VLLM_FUSED_MOE_CHUNK_SIZE=16384 #default 32768
export VLLM_CUSTOM_CACHE=1 #default 1
export VLLM_USE_OPT_CAT=1
export VLLM_USE_FUSED_FILL_RMS_CAT=1 #default 1(mtp 可能存在精度问题)
export VLLM_USE_LIGHTOP_MOE_SUM_MUL_ADD=0 #w4a16/awq # 需要关闭
export VLLM_USE_LIGHTOP_RMS_ROPE_CONCAT=0 # 不能和 kvfp8 一起开
export VLLM_USE_V32_ENCODE=1
export VLLM_USE_FLASH_MLA=1
export VLLM_DISABLE_DSA=0
export USE_LIGHTOP_TOPK=1
export USE_LIGHTOP_PER_TOKEN_GROUP_QUANT_FP8=1
```

ray：

```
#头节点
ray start --head --node-ip-address=10.16.1.116 --port=2766 --num-gpus=8 --num-cpus=16
#其他节点
ray start --address='10.16.1.116:2766' --num-gpus=8 --num-cpus=16
```

serve：

```
vllm serve /models/GLM-5-W8A8 --distributed-executor-backend ray \
    --disable-log-requests --trust-remote-code -pp 2 -tp 8 \
    --dtype bfloat16 --enable-chunked-prefill --max-model-len 48000 \
    --max-num-batched-tokens 8192 --max-num-seqs 64 --no-enable-prefix-caching
    --gpu-memory-utilization 0.9  \
    -cc '{"inductor_compile_config":{"combo_kernels": false}}' \
    --speculative_config '{"method": "mtp", "num_speculative_tokens": 2, "quantization":
"slimquant_marlin"}'
```

8.16. Minimax2.5

8.17. 参考

上面章节，我们是在海光的 Deepseek 推理实施的文档上构建更适配客户 PoC 的文档。

客户如果有需要也可以直接从海光的容器镜像上开始构建 deepseek 的推理环境。

截止：20251208 给的推荐安装方式。

```
# 文档里需要的物料可下面地址下载
wget -c https://taco-public-1251001002.cos.ap-
shanghai.myqcloud.com/external/haiguang/inference/Bw1000-llm-inference-
20251212.zip
```

如果需要上述测试的脚本文件，可以联系腾讯相关人员。